

实验报告

手写数字识别与垃圾分类

——AI 课程设计作业 3

计算机科学与技术学院

2451951　于沛可

2026 年 5 月 14 日

目录

1 问题概述 **3**

1.1 实验内容 3

1.2 数据集介绍 3

1.3 迁移学习理论背景 4

1.4 实验环境 4

1.5 实验要求 5

1.6 解决思路 5

2 模型设计 **6**

2.1 卷积神经网络基础 6

2.1.1 卷积运算 6

2.1.2 池化与批归一化 6

2.1.3 激活函数 7

2.2 LeNet-5 架构 7

2.3 MobileNetV2 架构 8

2.3.1 深度可分离卷积 8

2.3.2 倒残差结构 9

2.4 SE 注意力模块 9

2.5 LoRA 低秩适配 10

3 数据预处理与增强 **11**

3.1 MNIST 预处理 12

3.2 垃圾分类预处理与数据增强 12

3.3 数据增强对过拟合的缓解 13

4 微调策略 **13**

4.1 冻结微调 13

4.2 全量微调 14
4.3 LoRA 微调 15

5 实验结果与分析 16
5.1 LeNet-5 在 MNIST 上的表现 16
5.2 垃圾分类数据集概况 18
5.3 三种微调策略对比 18
5.3.1 全量微调分析 19
5.3.2 冻结微调分析 19
5.3.3 LoRA 微调分析 19
5.4 改进消融实验 20
5.4.1 关键发现：Baseline 优于 SE 变体 20
5.4.2 余弦学习率与标签平滑的作用 21
5.5 4 类 vs. 26 类对比分析 21
5.6 混淆矩阵分析 22
5.7 GPU 内存与训练效率分析 23
5.8 实验结果汇总 24

6 总结与展望 24
6.1 核心结论 24
6.2 对模型的理解 25
6.3 调试经验 25
6.4 未来改进方向 25

附录：项目代码结构 26

1 问题概述

1.1 实验内容

A. LeNet-5 + MNIST：深度学习入门实践

LeNet-5 + MNIST 被誉为深度学习的"Hello World"，是入门者学习图像分类的经典示例，有助于理解网络结构、数据加载、模型训练与评估的基本流程。

该任务的目的是了解深度学习基本流程，掌握 MindSpore 框架基本使用。任务内容是使用 MindSpore 框架构建 LeNet-5 神经网络，在 MNIST 手写数字数据集上训练模型，验证模型准确率。

B. MobileNetV2 + 垃圾分类数据集：模型微调实践

预训练模型能够显著加快训练速度，提升模型精度，并有效减少陷入局部最优解和过拟合的风险。微调（Fine-Tune）作为一种在预训练模型基础上调整参数以适应新任务的方法，已成为工业界广泛采用的策略。

该任务目的是掌握基于预训练模型进行 Fine-Tune，以适配特定领域任务。通过使用 MindSpore 加载 MobileNetV2 预训练模型，并在 26 类垃圾分类数据集上进行迁移学习与微调，最终评估模型在特定任务中的分类性能。

1.2 数据集介绍

MNIST 数据集是由 Yann LeCun 等人于 1998 年创建的手写数字灰度图像数据集 [1]，包含 60,000 张训练图像和 10,000 张测试图像，每张图像大小为 28×28 像素，涵盖 0–9 共 10 个类别。MNIST 的样本均经过尺寸归一化和居中处理，背景为黑色、前景为白色，类别分布基本均匀。在本实验中，MNIST 图像以三通道 RGB PNG 格式存储（三个通道数值相同），因此输入维度为 $3 \times 28 \times 28$。

垃圾分类数据集来源于华为 MindSpore 教程，包含 26 类常见生活垃圾的彩色照片。训练集共计 2,593 张图像，每类约 100 张；验证集共计 260 张图像，每类 10 张。26 类垃圾涵盖了厨余垃圾（如香蕉皮、茶叶渣）、可回收物（如塑料瓶、纸板箱、易拉罐）、有害垃圾（如电池、药瓶）和其他垃圾（如陶瓷碎片、一次性餐具）等多个大

类。该数据集的主要挑战在于：(1) 数据量极小，每类仅约百张训练样本；(2) 类间视觉相似度高；(3) 图像背景多样，拍摄条件不一致。图 8 展示了训练集各类样本的数量分布。

在 4 类分类实验中，原始 26 类被合并为可回收物（Recyclable）、有害垃圾（Hazardous）、厨余垃圾（Kitchen）和其他垃圾（Other）四个大类，以研究类别粒度对分类性能的影响。

1.3 迁移学习理论背景

迁移学习（Transfer Learning）[7] 是一种将已有任务（源域）上学到的知识迁移到新任务（目标域）上的机器学习范式。其核心思想在于：深度卷积网络的浅层倾向于学习通用的视觉特征（如边缘、纹理、颜色分布），这些特征对大多数视觉任务具有普适性；而深层特征则逐渐变得与具体任务相关。因此，利用在大规模数据集（如 ImageNet，包含 1,400 万张图像、1,000 个类别）上预训练的权重作为初始化，再在小规模目标数据集上进行微调，可以有效解决小样本场景下的训练困难。

在实践中，迁移学习的微调策略可分为以下几种层次：

- **冻结微调（Feature Extraction）**：冻结预训练骨干网络的全部参数，仅训练新添加的分类头。
- **全量微调（Full Fine-tuning）**：解冻全部参数，以较小的学习率更新骨干网络、以较大的学习率更新分类头。
- **参数高效微调（Parameter-Efficient Fine-tuning）**：如 LoRA[4]、Adapter 等方法，在冻结主干权重的同时插入少量可训练参数。

1.4 实验环境

本实验的软硬件配置如表 1 所示。

表 1 实验环境配置

项目	配置
操作系统	Ubuntu 22.04 LTS
GPU	NVIDIA GeForce RTX 4090 (24 GiB)
CUDA 版本	12.1
Python 版本	3.9.19
深度学习框架	MindSpore 2.4.1（GPU）
批量大小（MNIST）	128
批量大小（垃圾分类）	32
随机种子	42（固定以保证可复现性）

1.5 实验要求

本实验要求：(1) 成功复现并运行开源项目，详细说明代码原理与算法流程；(2) 在此基础上编写类似代码，完成模型加载与微调；(3) 使用冻结微调、全量微调、LoRA 微调等不同方式，说明工作原理，指出冻结了哪些层，并做显存/准确率/训练时间的对比；(4) 对模型进行改进以获得更高准确率，保证改进前后超参数一致并对比。

1.6 解决思路

第一阶段使用 MindSpore 数据加载接口处理 MNIST，完成归一化与维度调整；手写 LeNet-5 各层并配置交叉熵损失和 Adam 优化器，观察训练曲线验证流程正确性。

第二阶段：首先理解 MobileNetV2 结构并加载 ImageNet 预训练权重；然后将图像预处理为 224×224 RGB，替换分类头输出为 26 类；最后分别实现冻结微调（仅训练头部）、全量微调（骨干与头部同时训练，使用分组学习率）和 LoRA 适配（在 1×1 卷积层插入低秩矩阵）三种策略进行对比实验。在此基础上，通过引入 SE 注意力模块、余弦退火学习率调度和标签平滑等技术进行消融实验。

2 模型设计

2.1 卷积神经网络基础

卷积神经网络（CNN）[1] 通过三个核心设计实现高效特征提取：**局部感受野**——卷积核仅关注局部区域，提取边缘、纹理等特征；**权重共享**——同一卷积核在整个特征图上复用，赋予平移不变性；**空间子采样**——池化操作降低特征图维度，增强鲁棒性。

2.1.1 卷积运算

卷积层对输入特征图 X 施加卷积核 W 和偏置 b，经激活函数 σ 生成输出：

$$O_{ij}^{(k)} = \sigma\Big(\sum_{m=1}^{M} W_m^{(k)} * X^{(m)} + b^{(k)}\Big) \tag{1}$$

对于输入尺寸 $H_{\text{in}} \times W_{\text{in}}$、卷积核尺寸 $k \times k$、步幅 s、填充 p，输出特征图尺寸为：

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - k}{s} \right\rfloor + 1 \tag{2}$$

以 LeNet-5 的 C1 层为例：输入 28×28，卷积核 5×5，步幅 $s = 1$，无填充 $(p = 0)$，则 $H_{\text{out}} = \lfloor (28 + 0 - 5)/1 \rfloor + 1 = 24$，输出为 $6 \times 24 \times 24$。卷积层的参数量计算公式为：

$$\text{Params}_{\text{conv}} = (k \times k \times C_{\text{in}} + 1) \times C_{\text{out}} \tag{3}$$

2.1.2 池化与批归一化

池化层在 $k \times k$ 窗口内做下采样（最大池化或平均池化），在降低计算量的同时保留最显著的特征响应。池化操作不引入可训练参数。

批归一化（Batch Normalization, BN）[8] 通过对每个 mini-batch 内的特征进行

标准化，缓解内部协变量偏移：

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \tag{4}$$

其中 $\mu_{\mathcal{B}}$ 和 $\sigma_{\mathcal{B}}^2$ 分别为 mini-batch 的均值和方差，γ 和 β 为可学习参数。在 MobileNetV2 中，每个卷积层后均跟随 BN 层。

2.1.3 激活函数

Tanh：LeNet-5 使用的激活函数，输出范围 $(-1, 1)$，零中心有利于梯度传播，但深层时易梯度消失：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{5}$$

ReLU：现代 CNN 最常用的激活函数 [6]，计算简单且有效缓解梯度消失：

$$\mathrm{ReLU}(x) = \max(0, x) \tag{6}$$

ReLU6：MobileNetV2 使用的激活函数 [2]，上界 6 使激活值在低精度运算中更稳定：

$$\mathrm{ReLU6}(x) = \min(\max(0, x), 6) \tag{7}$$

输出层通过 Softmax 生成分类概率 $P(y_i) = e^{z_i} / \sum_j e^{z_j}$，训练时最小化交叉熵损失 $L = -\sum_i t_i \log P(y_i)$。

2.2 LeNet-5 架构

LeNet-5 是 LeCun 等人于 1998 年提出的经典 CNN[1]，是现代卷积网络的奠基之作。其网络结构如图 1 所示。

各层参数配置如表 2 所示。激活函数选用 tanh（与原文一致），总可训练参数为 44,726 个。

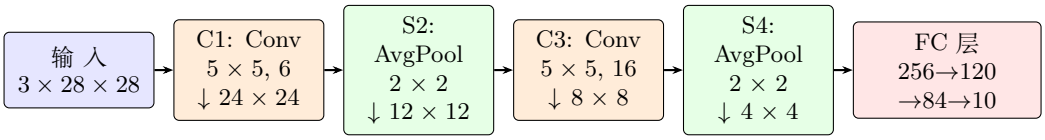


图 1 LeNet-5 网络结构示意图（RGB MNIST 输入，激活函数 Tanh）[1]

表 2 LeNet-5 各层参数配置

层名	操作	核大小/步幅	输出维度	参数量
Input	–	–	$3 \times 28 \times 28$	0
C1	卷积 + Tanh	5×5 / 1	$6 \times 24 \times 24$	456
S2	平均池化	2×2 / 2	$6 \times 12 \times 12$	0
C3	卷积 + Tanh	5×5 / 1	$16 \times 8 \times 8$	2,416
S4	平均池化	2×2 / 2	$16 \times 4 \times 4$	0
F5	全连接 + Tanh	–	120	30,840
F6	全连接 + Tanh	–	84	10,164
Output	全连接	–	10	850
总计（全部可训练）				**44,726**

2.3 MobileNetV2 架构

MobileNetV2[2] 是 Google 提出的轻量级 CNN，核心创新是**深度可分离卷积**和**倒残差结构**。

2.3.1 深度可分离卷积

深度可分离卷积 [3] 将标准卷积分解为两步：(1) 深度卷积（Depthwise）——对每个输入通道独立施加 $k \times k$ 空间滤波器；(2) 逐点卷积（Pointwise）——用 1×1 卷积融合通道信息。如图 2 所示。

标准卷积的计算量为 $D_K^2 \cdot M \cdot N \cdot D_F^2$，深度可分离卷积将其分解为：

$$\text{Cost}_{\text{DW}} = D_K^2 \cdot M \cdot D_F^2, \quad \text{Cost}_{\text{PW}} = M \cdot N \cdot D_F^2 \tag{8}$$

计算量比值为：

$$\frac{\text{Cost}_{\text{DW}} + \text{Cost}_{\text{PW}}}{\text{Cost}_{\text{std}}} = \frac{1}{N} + \frac{1}{D_K^2} \tag{9}$$

当 $D_K = 3$、N 远大于 1 时，计算量约为标准卷积的 $1/8 \sim 1/9$，参数量同样大幅减少。

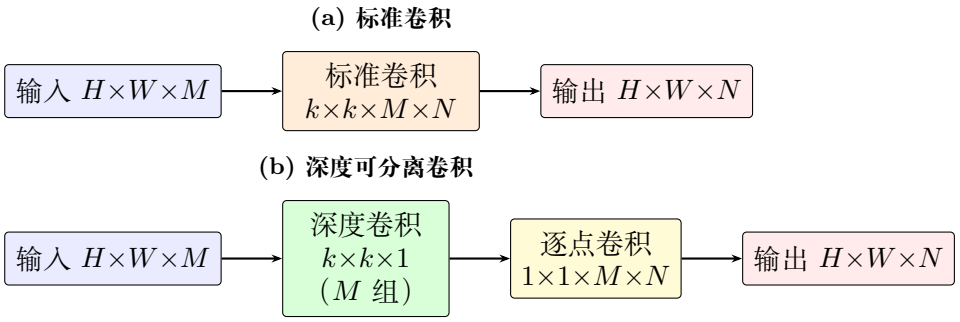


图 2 标准卷积与深度可分离卷积的计算流程对比 [3]

2.3.2 倒残差结构

倒残差结构（Inverted Residual）[2] 与 ResNet 的瓶颈结构相反，采用”两头窄中间宽”的设计：先用 1×1 卷积将通道从 k 扩展到 tk（扩展因子 t），再用 3×3 深度卷积进行空间滤波，最后用 1×1 卷积将通道压缩回 k'。残差连接在窄的瓶颈层之间建立。如图 3 所示。

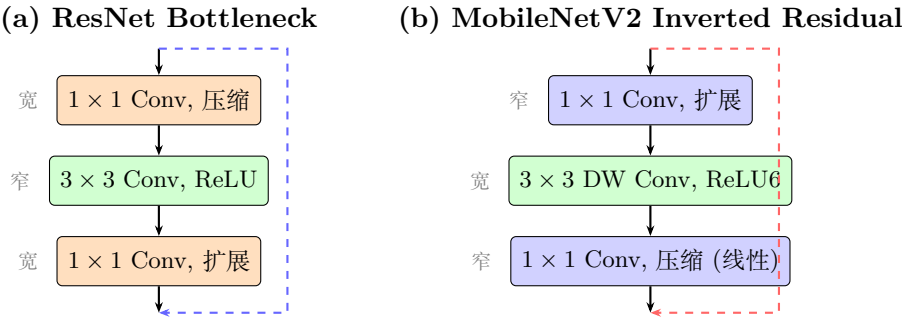


图 3 ResNet 瓶颈块与 MobileNetV2 倒残差块结构对比。虚线表示残差连接 [2, 6]。

MobileNetV2 在压缩层后使用线性映射（不加 ReLU6），因为低维空间中非线性会导致信息损失 [2]。网络整体结构如表 3 所示。

MobileNetV2 的总参数量约为 2,291,290，其中骨干网络（Backbone）约 225.8 万参数，分类头约 33,306 参数（$1280\times 26+26$）。

2.4 SE 注意力模块

SE（Squeeze-and-Excitation）[5] 通过显式建模通道间依赖关系来增强网络的表征能力。其结构如图 4 所示，包含三个步骤：

表 3 MobileNetV2 网络架构 [2]。t: 扩展因子，c: 输出通道数，n: 重复次数，s: 首块步幅。

输入尺寸	算子	t	c	n	s
$224^2 \times 3$	Conv2d 3×3	–	32	1	2
$112^2 \times 32$	Bottleneck	1	16	1	1
$112^2 \times 16$	Bottleneck	6	24	2	2
$56^2 \times 24$	Bottleneck	6	32	3	2
$28^2 \times 32$	Bottleneck	6	64	4	2
$14^2 \times 64$	Bottleneck	6	96	3	1
$14^2 \times 96$	Bottleneck	6	160	3	2
$7^2 \times 160$	Bottleneck	6	320	1	1
$7^2 \times 320$	Conv2d 1×1	–	1280	1	1
$7^2 \times 1280$	AvgPool 7×7	–	–	1	–
$1^2 \times 1280$	Conv2d 1×1	–	k	1	–

1. **Squeeze**：对特征图 $U \in \mathbb{R}^{H \times W \times C}$ 做全局平均池化：

$$z_c = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} U_c(i, j) \tag{10}$$

2. **Excitation**：用两个全连接层学习通道注意力权重：

$$s = \sigma\big(W_2 \cdot \delta(W_1 \cdot z)\big), \quad W_1 \in \mathbb{R}^{C/r \times C},\ W_2 \in \mathbb{R}^{C \times C/r} \tag{11}$$

其中 δ 为 ReLU，σ 为 Sigmoid，r 为缩减比（本文取 $r = 4$）。

3. **Scale**：将注意力权重逐通道乘回原始特征图：$\tilde{X}_c = s_c \cdot U_c$。

在本项目中，SE 模块被插入到每个 MobileNetV2 倒残差块的末尾（1×1 线性投影之后、残差加法之前）。

2.5 LoRA 低秩适配

LoRA（Low-Rank Adaptation）[4] 在指定的权重矩阵旁路插入低秩分解，冻结原始权重 W，仅训练低秩矩阵 A, B：

$$W' = W + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k) \tag{12}$$

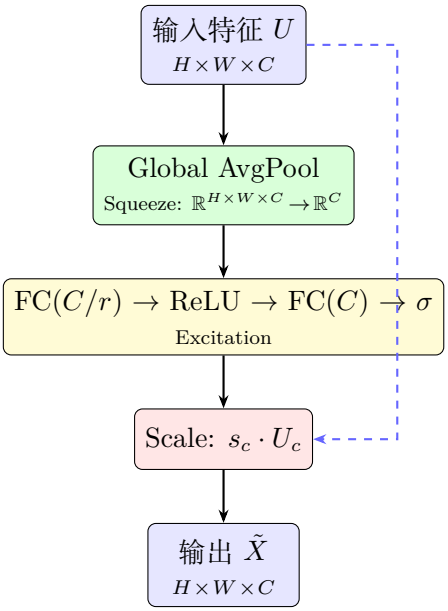


图 4 Squeeze-and-Excitation（SE）模块结构 [5]。虚线表示从输入到 Scale 的直连。

其原理如图 5 所示。原始权重在前向传播中保持不变，仅低秩旁路参与梯度更新，显著减少可训练参数量。

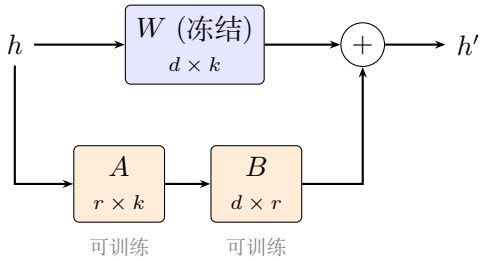


图 5 LoRA 低秩适配原理 [4]：冻结原权重 W，通过低秩矩阵 A, B 的旁路学习增量。B 初始化为零。

在本项目中，LoRA 模块被应用于 MobileNetV2 每个倒残差块中的 1×1 逐点卷积层。A 使用 Kaiming 正态初始化，B 初始化为零矩阵，确保训练开始时 LoRA 旁路输出为零。缩放因子 α/r 控制更新幅度，本实验取 $\alpha = 16$。

3 数据预处理与增强

数据预处理和增强策略对小样本学习的效果至关重要。本实验中 MNIST 和垃圾分类两个任务采用了不同的预处理管线。

3.1 MNIST 预处理

MNIST 图像为 28×28 灰度图像，以三通道 RGB PNG 格式存储。预处理流程：

```
transforms_mnist = [
    vision.Resize((28, 28)),
    vision.Rescale(1.0 / 255.0, 0.0),  # [0,255] -> [0,1]
    vision.Normalize(mean=[0.1307]*3,
                     std=[0.3081]*3),     # MNIST 全局统计量
    vision.HWC2CHW()                      # -> (C,H,W) 格式
]
```

Listing 1 MNIST 数据预处理管线

`Normalize` 使用 MNIST 训练集的全局均值 0.1307 和标准差 0.3081 进行标准化，使输入分布近似 $\mathcal{N}(0,1)$。

3.2 垃圾分类预处理与数据增强

垃圾分类图像为尺寸不一的彩色照片，需统一到 224×224 分辨率以匹配 MobileNetV2 的输入要求。

```
train_transforms = [
    vision.RandomResizedCrop(
        size=224, scale=(0.7, 1.0),
        ratio=(3./4., 4./3.)),  # 尺度+位置增强
    vision.RandomHorizontalFlip(prob=0.5),
    vision.RandomRotation(degrees=30),
    vision.Rescale(1.0 / 255.0, 0.0),
    vision.Normalize(mean=[0.485, 0.456, 0.406],
                     std=[0.229, 0.224, 0.225]),
    vision.HWC2CHW()
]
val_transforms = [
    vision.Resize(256),
    vision.CenterCrop(224),
    vision.Rescale(1.0 / 255.0, 0.0),
```

```
    vision.Normalize(mean=[0.485, 0.456, 0.406],
                     std=[0.229, 0.224, 0.225]),
    vision.HWC2CHW()
]
```

Listing 2 垃圾分类训练集预处理与增强

训练集的数据增强包含三个核心操作：

RandomResizedCrop：在 $[0.7, 1.0]$ 面积比例和 $[3/4, 4/3]$ 宽高比范围内随机裁剪并缩放到 224×224，同时实现尺度增强和位置增强。

RandomHorizontalFlip：以 50% 概率水平翻转，大多数垃圾物品左右翻转后语义不变，可将有效样本量翻倍。

RandomRotation：在 $[-30°, +30°]$ 范围内随机旋转，增强模型对旋转的不变性。

验证集使用确定性的 `Resize(256) + CenterCrop(224)` 策略，确保评估可复现。Normalize 使用 ImageNet 统计量 $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$，与预训练模型的输入分布保持一致。

3.3 数据增强对过拟合的缓解

在仅有 2,593 张训练图像的约束下，数据增强是缓解过拟合最直接的手段。然而，即使使用了上述增强策略，全量微调仍然出现严重过拟合（训练准确率 100% vs. 验证准确率 ~77%–80%）。这表明仅靠几何变换级别的增强无法从根本上解决数据不足的问题，更高级的增强方法（如 Mixup[9]、CutMix）或半监督学习方法可能是进一步改善的方向。

4 微调策略

4.1 冻结微调

冻结微调将 MobileNetV2 的骨干网络参数全部冻结，仅训练分类头：

```
net = MobileNetV2(num_classes=26)
try_load_backbone_checkpoint(net, ckpt_path,
    skip_substrings=("head", "classifier", "dense"))
# 冻结骨干：17个倒残差块 + 首尾卷积
for p in net.backbone.get_parameters():
    p.requires_grad = False
# 仅训练分类头
for p in net.head.get_parameters():
    p.requires_grad = True
optimizer = nn.Adam(
    filter(lambda x: x.requires_grad,
           net.get_parameters()),
    learning_rate=5e-4)
```

Listing 3 冻结微调核心逻辑

冻结的参数包括 17 个倒残差块及首尾卷积层中的全部约 225.8 万参数，仅训练分类头的 $1280 \times 26 + 26 = 33{,}306$ 个参数。训练 50 轮后达到最佳验证准确率 **65.38%**。

4.2 全量微调

全量微调解冻所有参数，骨干和头部使用分组学习率：

```
backbone_params = list(filter(
    lambda x: 'backbone' in x.name,
    net.get_parameters()))
head_params = list(filter(
    lambda x: 'head' in x.name,
    net.get_parameters()))
# 骨干小学习率保留预训练特征
# 分类头大学习率快速适配
group_params = [
    {'params': backbone_params, 'lr': 2e-5},
    {'params': head_params, 'lr': 3e-4}
]
optimizer = nn.Adam(group_params,
```

```
                    weight_decay=3e-4)
```

Listing 4 全量微调分组学习率配置

全量微调使所有 2,291,290 个参数可训练，训练 100 轮后达到最佳验证准确率 **77.31%**。配合余弦学习率退火（cosine annealing）和线性预热（warmup），学习率在前 5 轮从零线性增长至设定值，之后按余弦曲线衰减至接近零：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{t\pi}{T}\right) \tag{13}$$

4.3 LoRA 微调

LoRA 在冻结骨干权重的同时，在 1×1 卷积层旁路插入低秩矩阵：

```
class LoRAConv1x1(nn.Cell):
    def __init__(self, original_conv, rank=8, alpha=16):
        super().__init__()
        self.original_conv = original_conv
        for p in self.original_conv.get_parameters():
            p.requires_grad = False  # 冻结原始权重
        in_ch = original_conv.in_channels
        out_ch = original_conv.out_channels
        self.scaling = alpha / rank
        # A: Kaiming初始化; B: 零初始化
        self.lora_A = nn.Dense(in_ch, rank, has_bias=False)
        self.lora_B = nn.Dense(rank, out_ch, has_bias=False)
        self.lora_B.weight.set_data(
            initializer(Zero(), self.lora_B.weight.shape))

    def construct(self, x):
        orig_out = self.original_conv(x)
        B, C, H, W = x.shape
        x_flat = x.transpose(0, 2, 3, 1).reshape(-1, C)
        lora_out = self.lora_B(self.lora_A(x_flat))
        lora_out = lora_out.reshape(B, H, W, -1)
```

```
        lora_out = lora_out.transpose(0, 3, 1, 2)
        return orig_out + lora_out * self.scaling
```

Listing 5 LoRA 适配层核心实现

LoRA 实验结果：

- **LoRA r=8**：可训练参数 45,034 / 总参数 2,303,018，验证准确率 **54.23%**（100 epochs）
- **LoRA r=16**：可训练参数 56,762 / 总参数 2,314,746，验证准确率 **54.62%**（100 epochs）

LoRA 方法准确率较低的原因在于仅修改了极少参数（∼45K/2.3M ≈ 2%），低秩旁路的表达能力有限。但 LoRA 训练过程稳定，无剧烈波动，适合多任务部署场景——可为每个下游任务保存独立的小权重文件，共享一个冻结骨干。

5 实验结果与分析

5.1 LeNet-5 在 MNIST 上的表现

LeNet-5 在 MNIST 上训练 50 轮（使用余弦学习率 + warmup），验证准确率在第 48 轮达到最优 **98.54%**，训练准确率为 99.57%。模型总参数量为 44,726 个，全部可训练。

从训练曲线可以观察到：(1) 前 10 轮快速下降，模型快速学习基本手写数字特征；(2) 10–30 轮曲线趋于平稳，验证准确率在 98.0%–98.5% 之间波动；(3) 第 48 轮达到峰值 98.54%。整个训练过程中训练集与验证集表现接近，未出现明显过拟合——MNIST 的 60,000 张训练样本相对于 44,726 个参数是充足的（样本/参数比约 1,341:1）。

该结果与 LeCun 原文 [1] 报告的 98–99% 基本一致，验证了 LeNet-5 实现的正确性。图 7 展示了模型对 0–9 各数字的预测可视化。

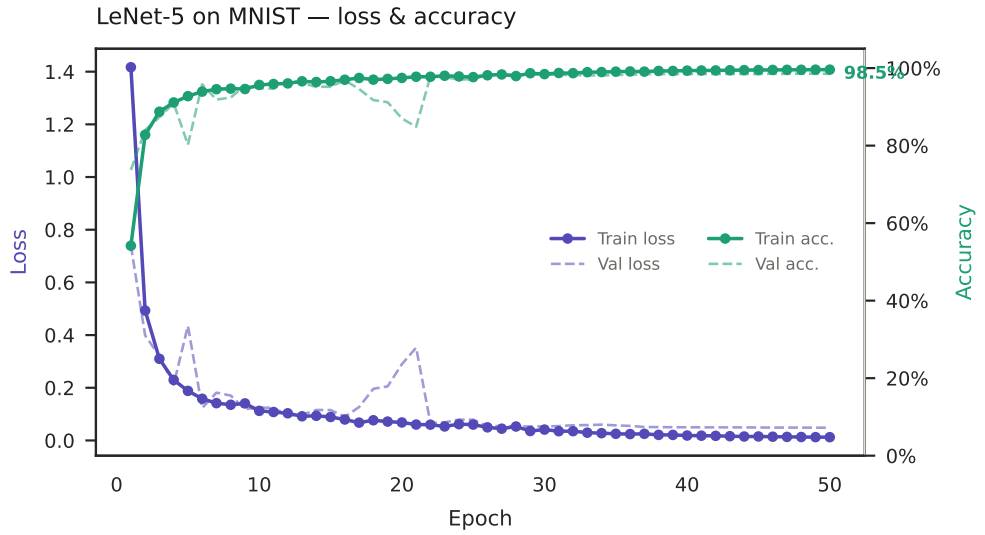


图 6　LeNet-5 在 MNIST 上的训练/验证损失与准确率曲线（50 epochs, cosine LR + warmup）

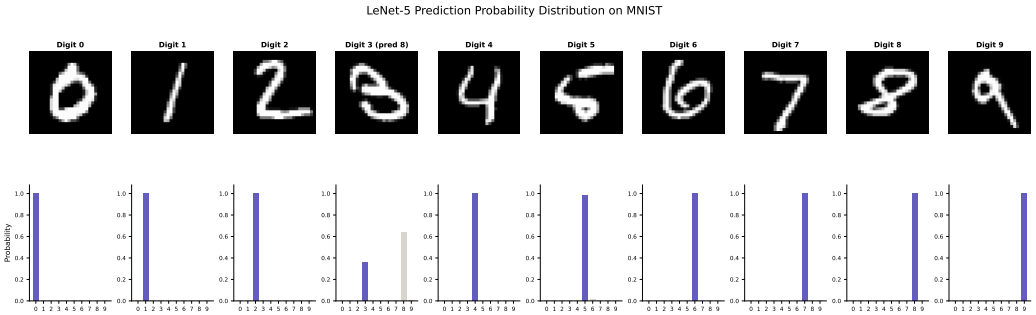


图 7　LeNet-5 对 0–9 各数字的预测概率分布（Softmax 输出）

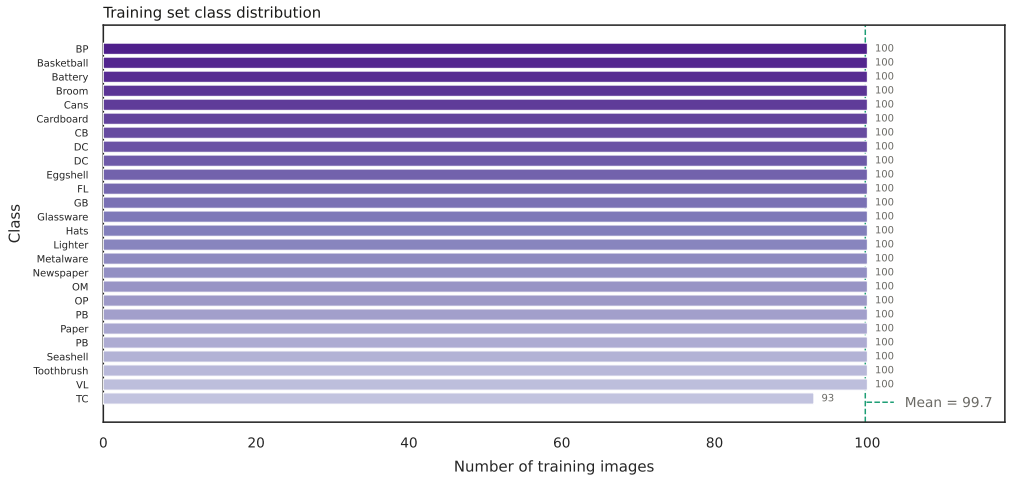


图 8　垃圾分类训练集各类样本数量分布（共 26 类，2,593 张）

5.2 垃圾分类数据集概况

该数据集每类训练样本约 100 张，总计 2,593 张训练图像和 260 张验证图像（每类 10 张）。需要特别注意的是，验证集中每个 0.38% 的准确率变化仅对应 1 张图片的分类差异（$1/260 \approx 0.38\%$），这使得各方法之间的微小差距（如 0.38%–1%）缺乏统计显著性。

5.3 三种微调策略对比

在相同 ImageNet 预训练初始化下，对冻结微调、全量微调和 LoRA 进行系统对比：

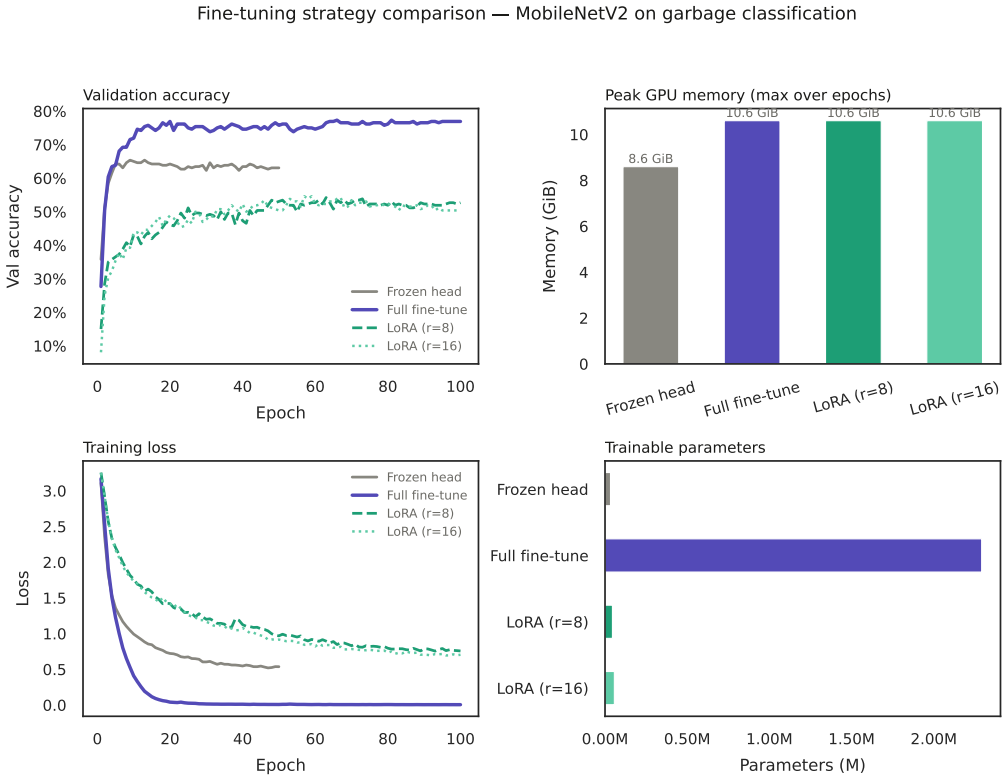


图 9 三种微调策略训练过程对比：验证准确率、训练损失曲线

表 4 三种微调策略的详细对比

策略	可训参数	总参数	训练轮数	最优 val_acc
冻结微调（仅 head）	33,306	2,291,290	50	65.38%
全量微调	2,291,290	2,291,290	100	**77.31%**
LoRA (r=8)	45,034	2,303,018	100	54.23%
LoRA (r=16)	56,762	2,314,746	100	54.62%

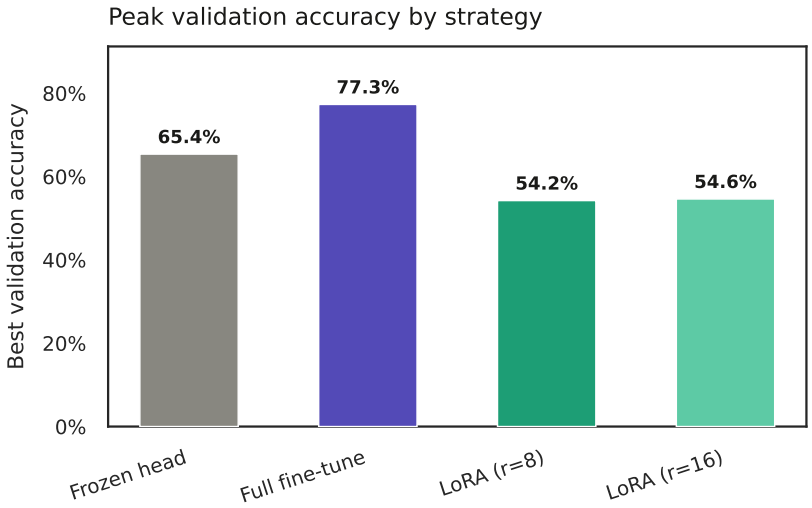


图 10 各微调策略最佳验证准确率对比

5.3.1 全量微调分析

全量微调训练 100 轮，所有 2,291,290 个参数可训练，达到最佳验证准确率 **77.31%**。骨干与分类头采用分组学习率 (2×10^{-5} / 3×10^{-4})，使骨干在保留 ImageNet 预训练特征的基础上缓慢适应目标域。

训练过程呈现典型的小样本过拟合特征：训练集准确率很快达到 100% 并保持不变，而验证集准确率在 76%–77% 之间波动，形成约 23 个百分点的过拟合间隙。这表明 77.31% 已接近当前数据量约束下的性能上限。

5.3.2 冻结微调分析

冻结微调仅训练 33,306 个分类头参数，50 轮后达到 **65.38%**。骨干参数完全冻结，模型只能利用 ImageNet 预训练的通用特征进行线性分类。其优势在于训练速度快、显存占用低，且不易过拟合。考虑到 26 类的随机基线仅为 $1/26 \approx 3.8\%$，65.38% 的准确率证明了 ImageNet 预训练特征的良好迁移性。

5.3.3 LoRA 微调分析

LoRA (r=8) 和 LoRA (r=16) 在 100 轮训练后分别达到 **54.23%** 和 **54.62%**。两种配置的表现接近，表明从 r=8 到 r=16 增加的额外参数（约 11,728 个）未带来显著提升。

LoRA 准确率较低的根本原因在于可训练参数占比极小（45K/2.3M ≈ 2%），低秩旁路的表达能力有限——它只能对原始权重做小幅度的增量修正，而垃圾分类与 ImageNet 的域差距需要更大幅度的权重调整。但 LoRA 的训练过程非常稳定，无剧烈波动，且显存占用与冻结微调相当。

5.4 改进消融实验

在全量微调的基础上，依次引入 SE 注意力模块、余弦学习率调度和标签平滑进行消融实验。所有实验均使用相同的数据增强策略（RandomResizedCrop + HorizontalFlip + Rotation）、Dropout、余弦 LR + warmup，训练 100 轮。

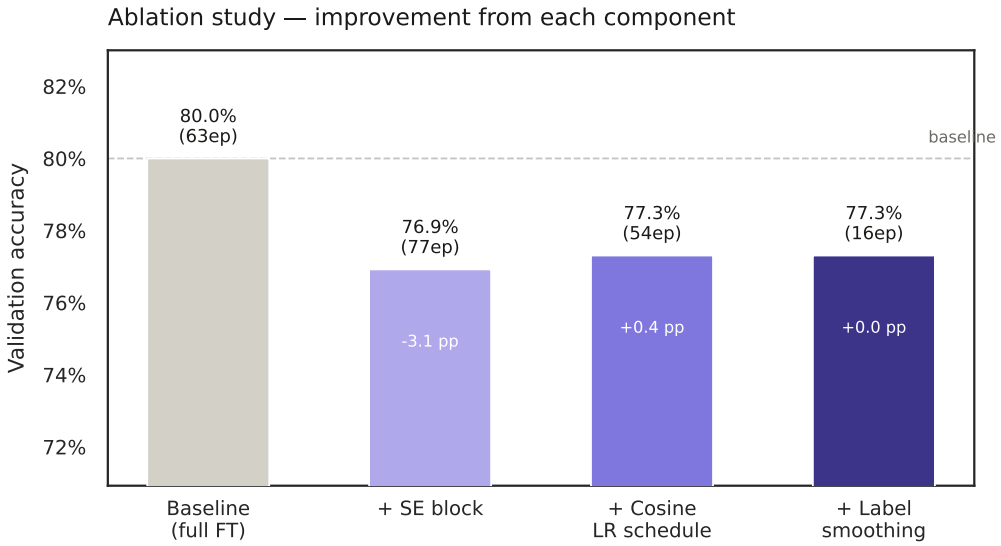


图 11 消融实验：各改进方法的验证准确率对比

表 5 MobileNetV2+SE 消融实验结果（26 类，100 epochs each）

配置	最优轮次	最优 val_acc	相比 Baselin
Baseline（无 SE，全量微调 + 强增强 +Dropout+cosine+warmup）	63	**80.00%**	–
+SE 注意力模块	77	76.92%	−3.08%
+SE + Cosine LR	54	77.31%	−2.69%
+SE + Cosine + Label Smoothing	16	77.31%	−2.69%

5.4.1 关键发现：Baseline 优于 SE 变体

消融实验最重要的发现是：**Baseline（无 SE 的全量微调 + 强数据增强 + Dropout + cosine LR + warmup）以 80.00% 的准确率成为 26 类任务的最佳**

配置，而所有引入 SE 模块的变体反而表现更差。

这一反直觉的结果可以从以下角度解释：

(1) 小数据集下 SE 增加过拟合风险：SE 模块为每个倒残差块引入额外的全连接层参数（$2 \times C \times C/r$ 个），在仅 2,593 张训练图像的约束下，增加的参数量加剧了过拟合。SE 块的通道注意力机制需要足够的数据多样性来学习有意义的通道权重分布，而小数据集无法提供这种多样性。

(2) 预训练特征已足够好：ImageNet 预训练的 MobileNetV2 已经具备良好的通道特征选择能力，在目标域数据不足的情况下，额外的 SE 通道注意力机制反而可能干扰已学习到的有效特征表示。

(3) 训练动态分析：Baseline 在第 63 轮达到最佳，而 +SE 变体在第 77 轮才达到次优的 76.92%——SE 模块增加了模型的有效容量，使得训练需要更长时间才能收敛，但最终仍然受限于数据量。

5.4.2 余弦学习率与标签平滑的作用

+SE+Cosine LR 和 +SE+Cosine+Label Smoothing 均达到 77.31%，略高于仅加 SE 的 76.92%。标签平滑（$\epsilon = 0.1$）通过将 one-hot 标签软化为 $[0.1/26, \ldots, 0.9 + 0.1/26, \ldots, 0.1/26]$，降低模型对标签的过度置信，具有正则化效果。值得注意的是，+SE+Cosine+LS 在第 16 轮就达到了 77.31%，说明标签平滑显著加快了收敛速度。

5.5 4 类 vs. 26 类对比分析

将 26 类垃圾合并为可回收物、有害垃圾、厨余垃圾、其他垃圾四大类后进行对比实验：

表 6 4 类 vs. 26 类垃圾分类对比

任务	方法	最优轮次	最优 val_acc
26 类	全量微调 (Baseline)	63	**80.00%**
26 类	+SE+Cosine+LS	16	77.31%
4 类	全量微调	82	80.38%
4 类	SE+Cosine+LS	16	79.62%

关键发现：4 类合并后准确率并未显著提升（80.38% vs. 80.00%），这一看似矛盾的结果有以下解释：

- **类内方差增大**：合并后每个大类包含多种视觉差异巨大的物品（如” 可回收物” 同时包含塑料瓶、纸板箱、金属罐等），类内方差的增大抵消了类别减少带来的简化。
- **数据量未增加**：合并类别不改变总训练样本数（仍为 2,593 张），数据量仍是根本限制。
- **验证集粒度**：4 类验证集同样仅 260 张，每个百分点的变化约对应 2.6 张图，统计波动显著。

5.6 混淆矩阵分析

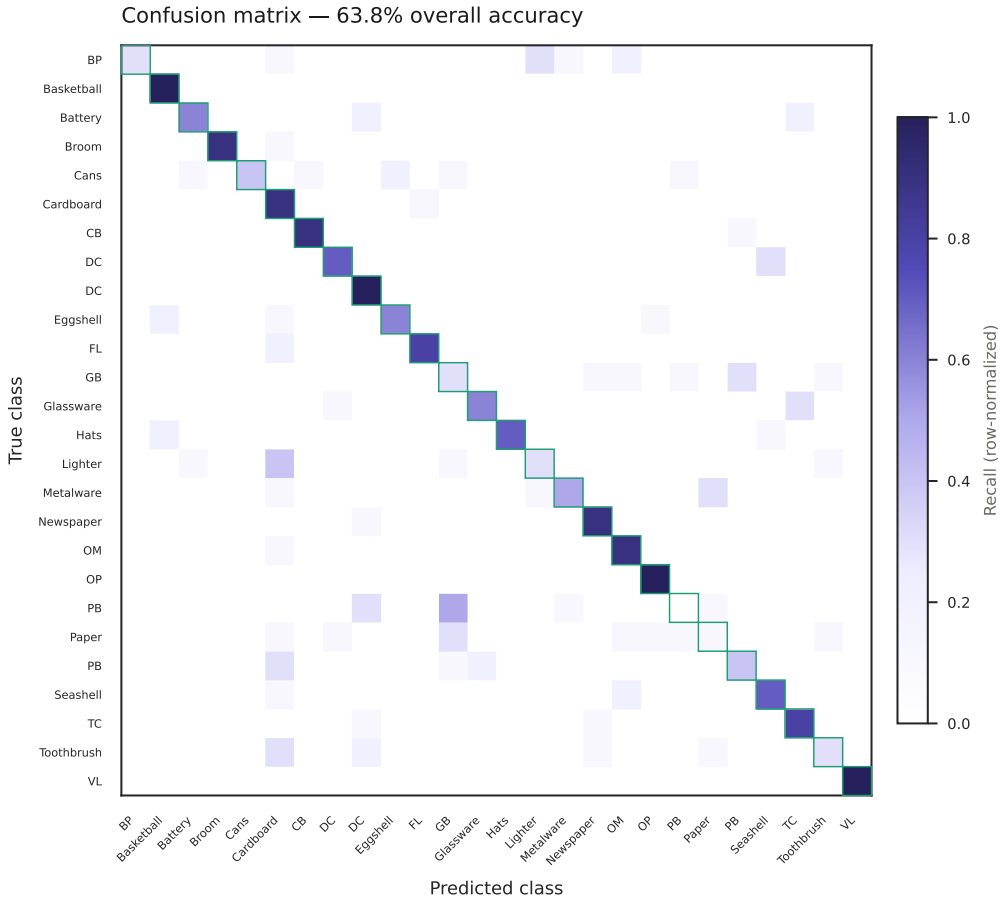


图 12 Baseline 模型（80.00%）在验证集上的混淆矩阵

混淆矩阵揭示的主要错误模式：

容器类混淆：不同类型的瓶子（塑料瓶、玻璃瓶、易拉罐）和盒子（纸盒、塑料盒）之间混淆率较高，因为它们在外观形态上高度相似，仅凭材质纹理区分难度大。

食物残渣类混淆：厨余垃圾中的不同种类（果皮、菜叶、剩菜）在颜色和形态上差异较小。

小样本波动：部分类别在验证集中仅有 10 张图像，1–2 张的预测变化即可导致该类准确率波动 10%–20%。

5.7 GPU 内存与训练效率分析

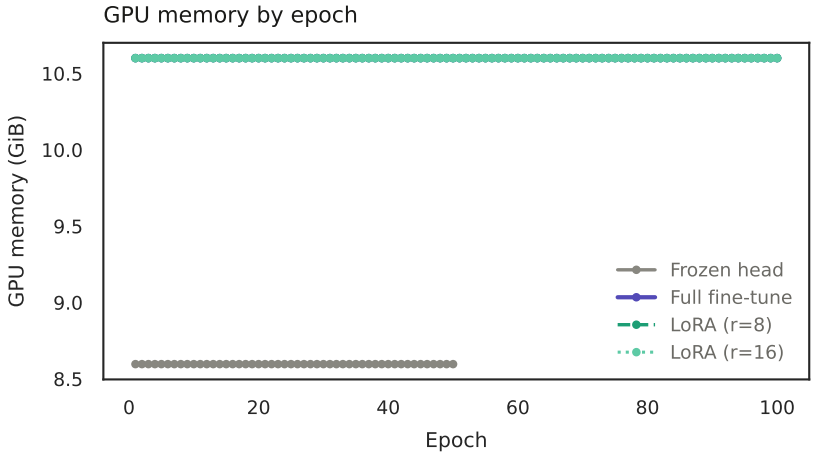


图 13 不同微调策略的 GPU 峰值显存占用对比

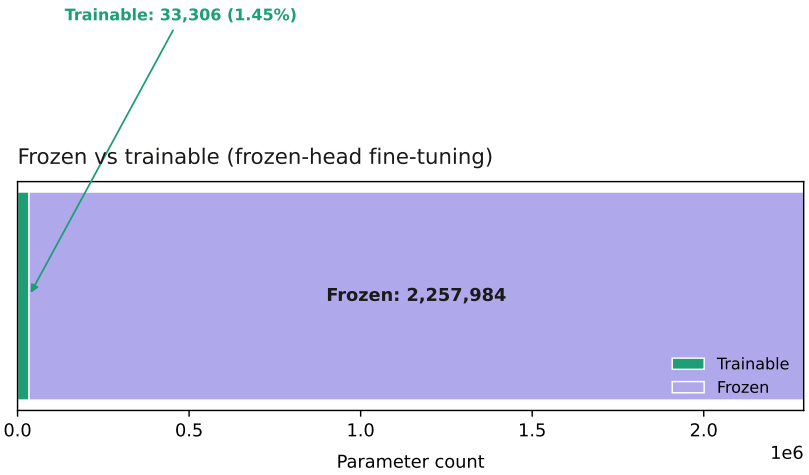


图 14 各微调策略的可训练参数量对比

冻结微调和 LoRA 的峰值显存显著低于全量微调。显存差异主要来源于：(1) Adam 优化器需为每个可训练参数维护一阶矩和二阶矩估计；(2) 反向传播需为每个

可训练参数保存梯度张量。全量微调的 229 万可训练参数带来了约 2 倍的额外显存开销。

5.8 实验结果汇总

表 7 全部实验结果汇总

任务	方法	可训参数	最优 val_acc	最优 epoch
MNIST (10 类)	LeNet-5	44,726	**98.54%**	48
26 类垃圾	冻结微调 (head only)	33,306	65.38%	–
26 类垃圾	全量微调	2,291,290	77.31%	–
26 类垃圾	LoRA r=8	45,034	54.23%	–
26 类垃圾	LoRA r=16	56,762	54.62%	–
26 类垃圾	Baseline (无 SE+ 全量 + 强增强)	2,291,290	**80.00%**	63
26 类垃圾	+SE	–	76.92%	77
26 类垃圾	+SE+Cosine LR	–	77.31%	54
26 类垃圾	+SE+Cosine+Label Smoothing	–	77.31%	16
4 类垃圾	全量微调	2,291,290	80.38%	82
4 类垃圾	SE+Cosine+LS	–	79.62%	16

6 总结与展望

6.1 核心结论

(1) 数据量是性能天花板的决定因素。在仅 2,593 张训练图像的约束下，无论使用何种模型改进技术，验证准确率均难以突破 80%。Baseline 全量微调（80.00%）已接近该数据量约束下的上限。SE 注意力模块在小数据集上反而降低了性能，说明在数据不足时盲目增加模型复杂度适得其反。

(2) 预训练质量比微调策略更重要。ImageNet 预训练提供了丰富的视觉特征，使全量微调达到 77%–80%；而 LoRA 因可调参数太少（∼2%），难以充分适配目标域。

(3) 类别粒度对准确率影响有限。4 类合并后（80.38%）与 26 类（80.00%）几乎持平，类别减少带来的简化被类内方差增大所抵消。

(4) 验证集统计波动显著。260 张验证图像中每张对应 0.38% 的准确率变化，使得方法之间 1%–2% 的差距缺乏统计显著性。

6.2 对模型的理解

LeNet-5：以仅 44,726 个参数在 MNIST 上达到 98.54%，展现了 CNN 的核心优势——局部感受野和权重共享。然而其容量有限：仅两个卷积层、tanh 激活在深层易梯度消失、全连接层占 93% 参数量。尽管如此，”卷积提特征 + 全连接做分类”的范式深刻影响了后续所有 CNN 设计。

MobileNetV2：通过深度可分离卷积（计算量约为标准卷积的 1/9）和倒残差结构，在移动端约束下实现精度与效率的平衡。线性瓶颈设计（压缩层后不加 ReLU6）基于”低维流形中非线性导致信息损失”的理论观察 [2]。

6.3 调试经验

布尔张量求和问题：MindSpore 中布尔张量需先转换为 `float32` 再求和，否则 `reduce_sum` 始终返回 0。

预训练权重参数名映射：MindSpore 官方 checkpoint 命名与自定义网络不一致，需编写专门的映射工具。对 SE 变体还需额外路径映射。

SE 模块初始化：将 SE 最后一层偏置初始化为 +3（$\text{sigmoid}(3) \approx 0.95$），使初始通道权重近似恒等变换。

6.4 未来改进方向

- **增加数据**：采用 Mixup[9]、CutMix 等高级增强策略，或收集更多真实样本。
- **更大模型**：在算力允许时尝试 ResNet-50[6]（∼2,560 万参数）或 EfficientNet。
- **LoRA 优化**：增大 rank（r=32/64）或将 LoRA 应用于 3×3 深度卷积层。
- **自监督预训练**：使用 SimCLR/MoCo 在目标域上无标签预训练，学习域内表示。

附录：项目代码结构

```
hw_dl/
  configs/config.yaml          -- 超参数配置
```

```
src/
  models/
    lenet5.py                -- LeNet-5 (RGB 三通道)
    mobilenetv2.py           -- MobileNetV2 + InvertedResidual
    mobilenetv2_se.py        -- MobileNetV2+SE (含 SE 注意力)
    lora.py                  -- LoRA 低秩适配模块
  trainers/
    train_mnist.py           -- LeNet-5/MNIST 训练
    finetune_frozen.py       -- 冻结微调
    finetune_full.py         -- 全量微调
    finetune_lora.py         -- LoRA 微调
    train_improved.py        -- 改进消融实验
  utils/
    pretrained.py            -- 预训练权重加载与映射
    visualizer.py            -- 可视化与图表生成
experiments/results.csv      -- 统一实验日志
run_all.sh                   -- 一键全流程脚本
```

参考文献

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *CVPR*, pp. 4510–4520, 2018.

[3] A. G. Howard, M. Zhu, B. Chen, *et al.* MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv:1704.04861*, 2017.

[4] E. J. Hu, Y. Shen, P. Wallis, *et al.* LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.

[5] J. Hu, L. Shen, and G. Sun. Squeeze-and-Excitation Networks. In *CVPR*, pp. 7132–7141, 2018.

[6] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *CVPR*, pp. 770–778, 2016.

[7] S. J. Pan and Q. Yang. A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.

[8] S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *ICML*, pp. 448–456, 2015.

[9] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond Empirical Risk Minimization. In *ICLR*, 2018.

[10] MindSpore Team. MindSpore: An Open Source Deep Learning Training/Inference Framework. https://www.mindspore.cn/, 2023.